从零开始 • 可独立动手

A 组从零手工复现教材

ISIC 2016 皮肤病灶分割 · Tiny U-Net · INT8 量化 · 黄金向量

> **学习目标** 读完并完成练习后，你应能自己下载和核验数据、编写数据处理与网络、训练与评估模型、实现 INT8 整数规则、导出并核对交接文件。

以 2026-09-23 项目实物为案例。面向没有机器学习、PyTorch 或 FPGA 基础的读者。

先读概念，再手算，再独立编程，最后运行真实数据并按关卡验收。

版本 1.0 | 科研与教学用途 | 不是临床诊断系统

学习路线与目录

建议顺序：先做概念与纸笔练习，再编程，最后运行真实数据。每章的“过关标准”用于自检。

学习路线与目录 2

使用本书之前：你将亲手造出什么 5

三种“自己实现”的层次 . . . 5

术语最短词典 . . . 5

第 1 章 为什么先找现成资源，再决定自建 6

1.1 项目需求如何变成筛选表 . . . 6

1.2 大模型与人工复核各做了什么 . . . 6

第 2 章 数据是实验的地基 7

2.1 什么是像素级分割 . . . 7

2.2 下载、SHA256 和安全解包 . . . 7

2.3 图像数组、通道和张量 . . . 8

第 3 章 预处理：把不同照片放进同一个方框 9

3.1 为什么不直接拉伸成正方形 . . . 9

3.2 翻转增强为什么要三张一起翻 . . . 9

第 4 章 划分：怎样防止“偷看考卷” 10

4.1 train、val、test 的三种职责 . . . 10

4.2 用一张表发现“分层”是否真的成功 . . . 11

第 5 章 卷积和 Tiny U-Net：模型到底在算什么 12

5.1 从一个 3×3 小算式开始 . . . 12

5.2 为什么网络像字母 U . . . 12

5.3 从网络图写出 PyTorch 代码 . . . 13

第 6 章 损失、梯度与训练：模型怎样学 14

6.1 logit、概率、预测掩膜 . . . 14

6.2 为什么同时用 BCE 和 soft Dice . . . 14

6.3 梯度下降、Adam、epoch、batch . . . 14

6.4 先用小图自检，再跑 40 轮 . . . 15

第 7 章 评估：0.87 到底代表什么 16

7.1 Dice 与 IoU 从集合算起 . . . 16

7.2 本次实际成绩与严格边界 . . . 16

第 8 章 为什么要把浮点模型变成 INT8 17

8.1 从真实数到整数格子 . . . 17

8.2 四步推导整数卷积 . . . 17

第 9 章 校准、完整前向和两类黄金向量 19

9.1 校准为什么只看训练集 . . . 19

9.2 黄金向量到底验证什么 . . . 19

9.3 二进制布局怎样由人手工检查 . . . 20

第 10 章 对比 FP32 与 INT8，读懂量化误差 21

10.1 要比较同一批图、同一个权重 . . . 21

10.2 发生精度下降时怎么找原因 . . . 21

第 11 章 测试、文件清单与工程交接 22

11.1 测试不等于再算一遍正常输入 . . . 22

11.2 容量预算怎样算，为什么它不是板级结果 . . . 22

11.3 项目目录导览：每个文件在流水线的位置 . . . 23

第 12 章 从空目录开始的独立实作实验 24

关卡 0：列实验合同 . . . 24

关卡 1：数据核验与配对 . . . 24

关卡 2：letterbox 与 Dataset . . . 24

关卡 3：固定划分 . . . 24

关卡 4：模型与算术参考 . . . 24

关卡 5：损失、指标与训练 . . . 24

关卡 6：独立测试 . . . 25

关卡 7：整数数学 . . . 25

关卡 8：字节级交接 . . . 25

关卡 9：比较并写诚实的结果 . . . 25

最终自测清单 . . . 25

第 13 章 常见卡点与排错树 26

命令行和环境 . . . 26

数据和形状 . . . 26

分数和量化 . . . 26

自己解释一个“为什么” . . . 26

附录 A：纸笔答案与公式速查 27

附录 B：复现旧结果的精确细节与修正方案 28

B.1 历史 split.json 如何产生 28
B.2 下一轮怎样做真正均衡的五档划分 28
B.3 校准名单与软件仿真的精确含义 28
附录 C：出处、许可证与进一步阅读 **29**

使用本书之前：你将亲手造出什么

你负责的是分工文件中的 **A 组：算法与量化。**最终目标不是写一篇漂亮的理论报告，而是形成可重复运行、可交给 B 组复核的真实文件。本书按本机 2026-09-23 已完成的工作顺序教学。纸笔练习只需要加减乘除；正式训练需要 Python、PyTorch 和 ISIC 数据。任何一步看不懂，先停下来运行那个小例子，不必一次记住所有术语。

完整流程只有一条主线：**找可信数据 → 验证文件 → 预处理 → 固定划分 → 搭网络 → 训练 → 独立评估 → INT8 量化 → 生成黄金向量 → 校验交付。**这里的“黄金”仅指我们事先算好的参考答案，不意味着临床真值，也不等于 FPGA 已经算对。

阶段	你手写或运行的东西	过关时能看到什么
数据	下载、校验、配对和划分	900 对训练、379 对测试；720/180
FP32	Tiny U-Net、损失、训练脚本	最佳权重、逐轮日志、独立测试报告
INT8	尺度、整数卷积、Q24、导出	15 层二进制文件与逐层参考输出
检查	单元测试、哈希、逐层重算	92 项测试通过，15 层导出验证

> **边界** 本书重现的是 A 组电脑端算法与数值接口。B 组 RTL、C 组 PYNQ 系统集成、板级帧率与资源占用尚无实测。本系统仅用于课程与科研，不用于临床诊断。

三种“自己实现”的层次

第一层是能解释每个数字：比如为什么是 256×256、为什么训练 720 张、参数量为什么是 122113。第二层是看完原理后**不复制项目源码**，自己写 data.py、model.py、train.py、quant_math.py 等，并用本书的验收数字核对。第三层是能把一层卷积的输出用纸笔或几行 Python 独立算出来，确认 B 组硬件结果符合 A 组接口。完成三层才算“会做”。

本书中“现有项目”路径均相对于 E:/嵌赛/pld-medical。你可以把项目放到其他盘符；命令中的相对路径不必改。已有源码是**对照答案**，建议先遮住源码做练习，然后再阅读差异。

术语最短词典

图像是三维数字表，通常有红绿蓝三个通道；**掩膜**是每个像素的正确类别，病灶为 1、背景为 0；**模型**是把图像映射成预测掩膜的一组运算；**参数/权重**是训练会改变的数字；**训练**是根据错误修正权重；**验证**是训练过程中选择版本；**测试**是版本选定后一次性检查；**FP32**是 32 位浮点数；**INT8**是 8 位整数；**量化**是把浮点数近似映射到有限整数；**logit**是 sigmoid 前的实数评分；**黄金向量**是软件算出的已知输入和正确输出。

第 1 章 为什么先找现成资源，再决定自建

1.1 项目需求如何变成筛选表

任务资料提出“ISIC 2016、轻量 U-Net、最终在 PYNQ-Z2 上用 INT8 运行”。这三句话尚不足以确定一个仓库能直接用。一个现成文件还须同时满足：任务确为病灶**分割**而非分类；图像和掩膜来源清楚；网络运算可以由 B 组实现；训练划分可重复；有明确权重布局、尺度、舍入和黄金向量；许可证允许使用。缺一个关键项，就只能当参考，不能当交付物。

本次核查官方 ISIC 2016 Task 1 数据和多个开源仓库。官方数据可直接作为首版数据源；未找到一份同时符合全部交接要求的高质量成品。于是采用公开的 U-Net **设计思想**，自行实现简化网络和数值接口，没有复制第三方仓库源码。审查记录在 docs/09_A组开源资源核查与实施路线.md。

候选	可借鉴的内容	为什么不能直接当本项目交付
ISIC 官方 Task 1	图片、掩膜、官方测试集	还需预处理、划分、模型与 INT8 接口
UNET_ACCEL	PYNQ-Z2 / 量化硬件思路	网络、位宽、输入任务和本项目不一致
Pytorch-UNet	训练工程组织方法	原任务、许可证和硬件接口不吻合
MSDUNet	ISIC 研究模型	结构比本项目首版算子集合复杂

手工复现：资源核查卡

打开官方数据页 https://challenge.isic-archive.com/data/，找到 **2016 Task 1**，把四个下载链接、格式、数量和许可证抄进一张表。再打开任一 GitHub 候选仓库，查看模型代码、许可证文件、训练脚本、权重和导出格式；逐格标“满足/不满足/未知”。未知不能自己补成满足。只读 README 不够，因为 README 的通道数可能与模型代码不一致。

> **为什么** 先核查能避免把“在另一种数据上做过 U-Net”误当“已经满足本项目的 FPGA 数值协议”。这是需求工程，也是避免许可证和性能陈述出错的第一步。

1.2 大模型与人工复核各做了什么

本轮曾写 docs/prompts/A01_deepseek_基础实现.md，通过 DeepSeek Harness (dsh) 的 headless 配置生成 FP32 基础代码。配置里的 API 名称 deepseek-flash 对应 DeepSeek 官方 2026-09-10 发布的 V4.1 Flash。提示词明确输入输出、允许的算子、禁止的假成绩、测试范围和修改目录。**提示词是规格书，模型生成的代码只是待审稿。**随后人工检查数据泄漏、形状、有效区域损失、Windows 多进程等问题，修正并运行测试。INT8 导出和黄金向量是后续独立实现与复核。

如果你不用 dsh，照本书手写同样可以完成；工具不构成算法正确性的证明。你需要掌握的是：把要求写成可检查条目，逐项验证生成物，错误时定位到数据、数学或接口，而不是相信“代码能跑”。

练习 1

请用自己的话写五条提示词验收条件，其中至少包括“官方测试集不可训练”“输出 logits”“只能使用指定算子”“不得编造成绩”“缺掩膜必须报错”。对照上面的资源核查表，解释为什么一个公开仓库能训练不代表能交给 B 组。

第 2 章 数据是实验的地基

2.1 什么是像素级分割

分类只输出“这张图有没有病灶”；分割要对每个像素回答同一个问题。假设一张 4×4 图像，病灶真值掩膜是下面的 0/1 表，其中中间四格为病灶。模型最终也要输出一个同尺寸表。两表重合多少，决定 Dice 和 IoU。

```text
0 0 0 0
0 1 1 0
0 1 1 0
0 0 0 0
```

JPEG 图像不是训练标签。官方 PNG 掩膜才是老师给模型的“参考答案”。文件名 ISIC_0000000.jpg 应配 ISIC_0000000_Segmentation.png。配错一对，网络会学习错误关系。训练集和测试集的图像都可公开取得，测试集标签只用于最终评价。官方页面列出 900 张训练图及 900 张掩膜、379 张测试图及 379 张掩膜，并标注 CC0。[来源 1]

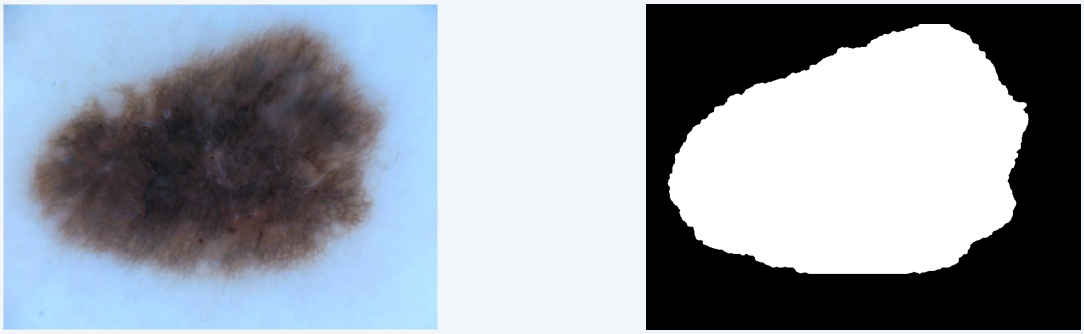


图 1 官方训练图 ISIC_0000000（左）及对应二值掩膜（右）。来源：ISIC 2016 Task 1，CC0。

2.2 下载、SHA256 和安全解包

四个官方 ZIP 很大，不要根据网页的四舍五入“MB”猜文件完整。**SHA256** 是文件内容的数字指纹：哪怕只改一个字节，哈希也几乎必然不同。它能检测下载损坏，也能记下数据版本；它不能证明数据标注医学上完全正确。项目的 data/raw/isic2016/provenance.json 保存四个原始文件的 URL、SHA256 和数量。

先在项目根目录建立 data/raw/isic2016/downloads，从官网 Task 1 四个链接分别下载并命名为 train_images.zip、train_masks.zip、test_images.zip、test_masks.zip。你可以在浏览器手动下载；项目中已有文件无需重复下载。PowerShell 查看哈希：

```powershell
Get-FileHash data/raw/isic2016/downloads/train_images.zip -Algorithm SHA256
Get-FileHash data/raw/isic2016/downloads/train_masks.zip -Algorithm SHA256
Get-FileHash data/raw/isic2016/downloads/test_images.zip -Algorithm SHA256
Get-FileHash data/raw/isic2016/downloads/test_masks.zip -Algorithm SHA256
```

本次下载记录的四个 SHA256 分别是 ad5b48fb...90ba2、81cff572...ddd876a、5cb5766a...13f63c99、065df24f...8492020e；完整 64 位值请读 provenance 文件。若你重新下载时不同，先确认官方文件是否更新，再考虑损坏；**不要强行把不同版本写成相同。**

解包可以手动做，但必须检查 ZIP 内文件路径，拒绝 ../、绝对路径和解包目录外写入。现有可对照的安全解包脚本是 software/role_a/extract_official.py：它核对每个 ZIP 的类型、数量、文件名和最终配对，再写来源清单。运行：

```powershell
py software/role_a/extract_official.py
```

手工重写这个脚本时，先实现三个纯函数：safe_member(name) 判定成员路径；extract_one(zip, out, expected) 只解出允许的 JPEG/PNG；pair_by_stem(images, masks) 按 ID 配对并报告缺失、重复、孤立文件。最后检查 900 对、379 对，训练与测试标识无交集。**发现异常立即失败**，不要静默跳过，这叫“让错误早暴露”。

> **过关标准** 目录 train/images 和 train/masks 各 900 个文件；test/images 和 test/masks 各 379 个。四个 SHA256 可追溯。源数据不要放进交接 ZIP。

2.3 图像数组、通道和张量

普通彩色图像可想作 [高度, 宽度, 3]：每个像素有 R、G、B 三个 0-255 整数。PyTorch 卷积默认使用 [批量 N, 通道 C, 高 H, 宽 W]。单张 256×256 RGB 在送入网络前是 [1,3,256,256]；一批四张是 [4,3,256,256]。原始像素除以 255，成为 0-1 的 FP32。掩膜要变为 [N,1,256,256] 且只有 0 或 1。

千万不要把掩膜也当普通照片做双线性缩放：0/255 边界会产生 63、127 等中间灰度，代表含糊标签。**照片用双线性插值**让边缘平滑，**掩膜用最近邻插值**保持离散类别。双线性本质是周围四点的加权平均；最近邻只取最近的一个原像素。

练习 2

假设原图是 2×2，左上黑、其余白；把掩膜放大 2 倍。你希望输出仍只有 0/1，应该用哪种插值？如果一个批次有 4 张 RGB 256×256 图，以 FP32 保存输入，共占多少字节？答案：最近邻；4×3×256×256×4 = 3,145,728 字节。

第 3 章 预处理：把不同照片放进同一个方框

3.1 为什么不直接拉伸成正方形

原图可能是 1024×768。若把宽高都硬拉到 256，圆形病灶会变形。这里用 **letterbox**：取缩放因子 r = 256 / max(W,H)，按比例缩放，再居中补黑边。对 1024×768，r=0.25，缩放后 256×192，上下各补 32 像素。左右没有补边。这样形状保留，但画布统一为 256×256。

```text
   1024x768 ->      256x192 ->      32 ->    256x256
valid:   =0       =1
```

valid 掩膜是第三张 0/1 表，用几何坐标生成：补边处 0、真实图像处 1。不能通过“图像像素是否为黑”判断 valid，因为原图本身可能有黑像素。训练损失和评价指标都只计算 valid=1 的区域，否则大量补边会人为抬高背景识别成绩。

你自己写的核心伪代码

```python
def letterbox(image, mask, size=256):
    W, H = image.size
    r = size / max(W, H)
    new_w, new_h = round(W * r), round(H * r)
    left = (size - new_w) // 2
    top = (size - new_h) // 2
    image_small = image.resize((new_w, new_h), BILINEAR)
    mask_small = mask.resize((new_w, new_h), NEAREST)
    canvas_image = zeros_rgb(size, size)
    canvas_mask = zeros_gray(size, size)
    valid = zeros_gray(size, size)
    paste(canvas_image, image_small, left, top)
    paste(canvas_mask, mask_small, left, top)
    fill(valid, left, top, new_w, new_h, value=1)
    return canvas_image, canvas_mask, valid
```

生产代码还要处理宽高无效、四舍五入溢出 1 像素、图像/掩膜原始尺寸不一致、EXIF 旋转，以及原始掩膜若非严格 0/255 时按 >=127 二值化并留记录。本次 900 张训练掩膜没有非二值异常，但检查仍要保留。对应实现见 software/role_a/data.py。

> **过关标准** 对一张 1024×768 的合成图，输出形状 256×256；valid 的 1 恰好覆盖行 32..223，边框为 0；掩膜仍只有 0/1；图像、掩膜和 valid 几何位置一致。

3.2 翻转增强为什么要三张一起翻

训练图像只有 720 张，随机水平/垂直翻转可以让模型看到更多方向。随机翻转必须对**图像、掩膜、valid 同步执行**，并且只在训练集启用。若只翻图像不翻掩膜，相当于给学生一张题纸配另一张答案。验证和测试不做随机增强，才能稳定比较不同轮次。

第 4 章 划分：怎样防止“偷看考卷”

4.1 train、val、test 的三种职责

官方提供 900 张训练图和 379 张测试图。本次从官方训练 900 张里，用固定种子 42 取 720 张训练、180 张验证。**训练集**参与梯度更新；**验证集**不参与梯度，用来选 best.pt；**官方测试集**在模型、阈值和量化规则确定后做独立报告。若测试集反复用于改网络，最终分数就不是独立估计。

原设计**打算**按原始掩膜的病灶像素比例分 5 档，再在档内随机分配，让大小病灶的分布接近。但这次历史实现有一个值得认真学习的缺陷：先从每档交替取样，再从汇总的验证列表尾部裁到 180 张，结果验证集只保留了面积最小的两档。对已保存的 900 张掩膜和 split.json 独立复算，五档各 180 张，其中验证数量依次是 **90、90、0、0、0**。因此本次训练/验证划分**可重复但不均衡**；它的验证 Dice 不能被描述为对全部病灶大小都具有代表性。固定 seed=42 和文件名排序只能保证重现同一划分，不能保证划分合理。

```python
pairs = sorted(all_900_pairs, key=lambda item: item.stem)
groups = bucket_by_lesion_fraction(pairs, bucket_count=5)
rng = Random(42)
train, val = [], []
for group in groups:
    rng.shuffle(group)
    val.extend(group[:36])       # 20% of each 180-item bucket
    train.extend(group[36:])
assert len(train) == 720 and len(val) == 180
assert set(ids(train)).isdisjoint(ids(val))
save_split_with_sorted_ids('split.json', train, val, seed=42)
```

上面的代码是**下一轮应采用的均衡分层示意**，不是这次历史代码的逐字节复制。项目的旧版分档和裁剪细节见附录 B 与 software/role_a/data.py。若想得到**逐字节一致**的旧版 split.json 和旧实验分数，必须复制它的缺陷性分配规则；仅写“种子 42”还不够。若要做可信的新实验，应先修正划分、重训、重校准、重测，并给新结果另起名字，不得把新划分和旧权重混报。运行旧实现的对照命令：

```powershell
py software/role_a/train.py prepare-split `
  --image-dir data/raw/isic2016/train/images `
  --mask-dir data/raw/isic2016/train/masks `
  --output-dir runs/role_a_split `
  --seed 42 --val-fraction 0.2
```

它生成 split.json、split.csv、split_report.json。另建 runs/role_a_split_with_test 只用于登记官方测试文件名；训练脚本若读到含 test 的划分，会直接拒绝。绝不能在同一目录覆盖训练划分。

```powershell
py software/role_a/train.py prepare-split `
  --image-dir data/raw/isic2016/train/images `
  --mask-dir data/raw/isic2016/train/masks `
  --output-dir runs/role_a_split_with_test `
  --seed 42 --val-fraction 0.2 --include-test `
  --test-image-dir data/raw/isic2016/test/images `
  --test-mask-dir data/raw/isic2016/test/masks
```

重要限制 本次旧版验证集仅含面积最小两档，是本轮 QA 发现的划分偏差；旧实验成绩须带这一条件解读。ISIC 2016 此次采用的公开文件名也没有足够的患者/病灶分组信息，本次 720/180 只是图像级划分，不能宣称患者级独立。随机种子不能保证 GPU 重训权重逐位相同。[来源 4]

练习 3

手写两个测试：一是同样文件和 seed 生成的 split.json 字节相同；二是故意把一个官方测试 ID 放到训练列表，程序必须拒绝。解释为什么“能读到测试图片”与“允许用它选最佳模型”是两回事。

4.2 用一张表发现“分层”是否真的成功

名称叫“分层划分”不代表结果真的分层。对 900 张原始掩膜逐个计算 area_ratio = 病灶像素数 / 全部像素数，按 (area_ratio, stem) 排序，每 180 张算一档，再用 split.json 统计各档的 train/val 数量。本机实测如下。最小面积档的范围从 1.155% 到 8.374%，最大面积档延伸到 99.544%；这反映源图像中病灶大小差异很大。

面积档（各 180 张）	原图病灶占比范围	旧版 train / val
1，最小	1.155%-8.374%	90 / 90
2	8.446%-16.113%	90 / 90
3	16.133%-27.488%	180 / 0
4	27.492%-42.182%	180 / 0
5，最大	42.254%-99.544%	180 / 0

这张表是“先算整体指标之前先查切片”的例子。720/180 总数正确，train/val 标识也不重叠，但验证集覆盖的面积范围仍不完整。你自己的划分脚本应在导出 JSON 的同时生成这张分档审计表，并让测试检查各档 val 是否接近 36。若下一轮按每档 36/144 划分，应得到 5 行 144 / 36；这仍需用真实文件重算确认。

第 5 章 卷积和 Tiny U-Net：模型到底在算什么

5.1 从一个 3×3 小算式开始

卷积可以先理解成一个会在图片上滑动的小模板。一个 3×3 模板有 9 个权重，当前位置的 9 个像素与它逐项相乘并相加，再加一个偏置。例：输入 3×3 像素全为 1，权重中心为 2、其他为 0，偏置为 -1，输出就是 1×2 + (-1) = 1。模板移动到每个位置，就形成一张新特征图。

对彩色输入，三个颜色通道各有一套 3×3 权重，全部求和才得到**一个输出通道**。若输出有 8 个通道，就有 8 套。权重张量布局叫 OIHW：输出通道 O、输入通道 I、卷积核高 H、卷积核宽 W。输入布局 NCHW，两者一定不要混淆。

第一层 3→8 的 3×3 卷积：权重数 8×3×3×3=216，偏置 8，共 **224 个可训练参数**。另一层 8→8：8×8×3×3+8=584。参数量与图片宽高无关；计算量与宽高有关。单层卷积乘加 MAC 约为 输出高×输出宽×输出通道×输入通道×核高×核宽，不把偏置加法算进 MAC。

padding=1 是在 3×3 卷积前周围补一圈零，让步幅 1 时输出宽高保持不变。1×1 卷积只把同一像素的多通道线性组合，本例最后用它把 8 通道变成 1 通道。ReLU(x)=max(0,x) 把负数置零。2×2 MaxPool 从四格取最大值，宽高减半；最近邻 Upsample 把一格复制到 2×2，宽高翻倍；Concat 沿**通道**拼接两张同宽高特征图。

手算练习 4

若输入单通道 3×3 全为 1，卷积核 3×3 全为 1，padding=1、偏置 0：中心输出 9，四角输出 4，四条边中点输出 6。请画出完整 3×3 输出矩阵；然后把它交给自己写的 conv2d 小函数核对。这个练习同时检验“补零”是否正确。

5.2 为什么网络像字母 U

U-Net 的左边逐步缩小宽高、增加通道，获得较大范围的上下文；右边逐步放大，并把左边同尺度的细节通过 **skip connection** 拼回来。原始 U-Net 论文提出收缩路径与扩张路径配合实现定位。[来源 2] 本项目是**更小的独立实现**，不是原论文网络原封不动照搬。

本项目首版刻意只用 Conv3×3、Conv1×1、ReLU、MaxPool2×2、最近邻 Upsample 和 Concat，方便 B 组逐个算子实现与核对。没有 BatchNorm、LayerNorm、Dropout、注意力、转置卷积、模型内 Sigmoid。限制算子不是说它们“科学上更优”，而是降低首版硬件算子和数值接口复杂度。

阶段	操作后张量 [N,C,H,W]	为什么要保留/改变
输入	[N,3,256,256]	RGB 统一尺寸
enc0	[N,8,256,256]	保存浅层细节供 skip
enc1	[N,16,128,128]	池化一次，增加通道
enc2	[N,32,64,64]	再扩大感受范围
enc3	[N,64,32,32]	最深特征
up2 + skip2	[N,96,64,64]	64+32 通道拼接
dec2	[N,32,64,64]	卷积压回 32 通道

阶段	操作后张量 [N,C,H,W]	为什么要保留/改变
up1 + skip1	[N,48,128,128]	32+16 通道拼接
dec1	[N,16,128,128]	逐级恢复
up0 + skip0	[N,24,256,256]	16+8 通道拼接
dec0 / head	[N,8,256,256] / [N,1,256,256]	最终每像素 1 个 logit

N 表示批量数，通常训练时 4。U 形中每个编码/解码块有两层 3×3 Conv，总共 4×2 + 3×2 + 1 = 15 层卷积。最后的 1 是 1×1 head。256 可被 $2^3=8$ 整除，因此三次池化和三次上采样能严格对齐；若输入宽高不是 8 的倍数，要有额外裁剪或补边规则。当前实现选择提前拒绝不合法尺寸。

5.3 从网络图写出 PyTorch 代码

先只做一个“双卷积块”，并给它写输入输出形状测试。然后组四个编码块、三次池化、三个解码块和 head。下面是**简化骨架**，让你看清关键连接；完整参数校验、变长 num_pools 和接口见 software/role_a/model.py。

```python
def block(cin, cout):
    return nn.Sequential(
        nn.Conv2d(cin, cout, 3, padding=1), nn.ReLU(),
        nn.Conv2d(cout, cout, 3, padding=1), nn.ReLU())

e0 = block(3, 8); e1 = block(8, 16)
e2 = block(16, 32); e3 = block(32, 64)
d2 = block(64 + 32, 32)
d1 = block(32 + 16, 16)
d0 = block(16 + 8, 8)
head = nn.Conv2d(8, 1, kernel_size=1)
```

前向必须先存 skip 再池化。在解码侧先上采样，再沿 dim=1 拼通道。**模型输出 logits，不在这里做 sigmoid**；sigmoid 留给损失和评价。原因是训练时 BCEWithLogits 用更稳定的合并实现。[来源 3]

```python
s0 = e0(x)
s1 = e1(pool(s0))
s2 = e2(pool(s1))
z  = e3(pool(s2))
z  = d2(torch.cat([upsample(z),  s2], dim=1))
z  = d1(torch.cat([upsample(z),  s1], dim=1))
z  = d0(torch.cat([upsample(z),  s0], dim=1))
logits = head(z)
```

验算总参数量时，每层使用 Cout×(Cin×K×K+1)，把 15 层相加得到 **122113**。纯 Python 参考计算器还给出每图卷积 **675282944 MAC**。这两个数都只是模型算术规模，不能推导出实际 FPGA 帧率或 LUT 数。你可运行：

```powershell
py software/role_a/verify_reference.py `
  --base-channels 8 --num-pools 3 --size 256
py software/role_a/train.py describe-model `
  --size 256 --base-channels 8 --num-pools 3
```

过关标准 喂入 [2,3,256,256] 输出 [2,1,256,256]；逐层形状和上表一致；总参数 122113；枚举算子没有额外类型。随机权重能前向运行仅表示结构连通，不代表模型已经学会分割。

第 6 章 损失、梯度与训练：模型怎样学

6.1 logit、概率、预测掩膜

网络最后输出每像素一个任意实数 z，叫 logit。sigmoid(z)=1/(1+exp(-z)) 将其变成 0-1 概率；z=0 对应 0.5。评价时以概率 >=0.5 判为病灶，因此也等价于 logit >=0。这是一条**固定的决策规则**，不是神奇的医学临界值。测试前若改阈值，必须只在验证集决定。

6.2 为什么同时用 BCE 和 soft Dice

BCE 对每个有效像素单独惩罚错误概率。真值 1 时希望预测概率高；真值 0 时希望它低。二分类交叉熵公式：-y log(p) - (1-y) log(1-p)。PyTorch 的 binary_cross_entropy_with_logits 接收 logit，在内部合并 sigmoid 和 BCE，数值上比自己先 sigmoid 再取 log 更稳定。[来源 3]

Dice 看的是整个病灶区域的重合。训练时硬阈值不可导，所以用概率代替预测的 0/1，形成 soft Dice：(2Σ(p*y)+ε)/(Σp+Σy+ε)。本项目 ε=1，损失是 BCE + (1-softDice)，两项权重都为 1。每项只在 valid=1 的像素上算；否则补黑边也会进入梯度。ε 防止全空时除零，并定义了平滑行为。

手算练习 5

两格有效像素，真值 [1,0]，预测概率 [0.8,0.2]。soft Dice 分子 2×0.8+1=2.6，分母 0.8+0.2+1+1=3，soft Dice 为 0.8667，soft Dice 损失约 0.1333。再加入一个 valid=0 的第三格，任意改它的预测值，损失应保持不变。这是代码测试的核心。

6.3 梯度下降、Adam、epoch、batch

模型有 122113 个参数。训练一步是：拿一批图片做前向 → 计算损失 → 反向传播得到每个参数的梯度 → 优化器沿降低损失方向微调参数。学习率 0.001 控制步幅；Adam 会记录梯度的一阶、二阶移动平均来调整不同参数的步幅。你无需先手推完整 15 层导数，但应知道 loss.backward() 得出梯度，optimizer.step() 改权重，optimizer.zero_grad() 清掉上一步梯度。

一个 **batch** 是一次看的图片数，本次为 4；一个 **epoch** 是把 720 张训练图完整看一遍，所以约有 720/4=180 次参数更新。共 40 epoch，约 7200 次更新。每轮结束让模型进入 eval()，在 180 张验证图上计算指标；验证过程用 no_grad()，不会改权重。若本轮验证平均 Dice 创新高，保存 best.pt。本次最佳在第 40 轮。

```python
for epoch in range(40):
    model.train()
    for image, mask, valid in train_loader:
        optimizer.zero_grad(set_to_none=True)
        logits = model(image)
        loss = bce_valid(logits, mask, valid) + dice_loss_valid(logits, mask, valid)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 5.0)
        optimizer.step()
    model.eval()
    with torch.no_grad():
        val_dice = mean_per_image_dice(model, val_loader)
    if val_dice > best_dice:
        save_checkpoint(model, epoch, val_dice)
```

clip_grad_norm_ 是防止极大梯度造成一次参数跳跃；本次上限 5。固定 Python、NumPy、PyTorch 的随机种子并保存配置有助于复现，但 GPU 某些实现与运行环境仍可能造成微小差异，不能承诺重训出的权重逐位相同。[来源 4]

6.4 先用小图自检，再跑 40 轮

新手第一次运行应先用 CPU、64×64、小通道和 1 轮检验整个流程能走通。这份 smoke 结果**不是正式精度**。正式运行再用 256×256、8 通道、40 轮和训练集同步翻转。依赖需 Python 3.10+、PyTorch、NumPy、Pillow。建议建虚拟环境隔离项目依赖；本机已有 .venv，不要为了阅读教材重新装 4 GB 环境。

```powershell
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install torch numpy Pillow
.\.venv\Scripts\python.exe -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

上面是**新机器**的一般命令；PyTorch CUDA 安装方式和版本应按官方安装页选你的驱动与平台。[来源 8] 如果你只是重用本项目，先测试现有 .venv/Scripts/python.exe，不要重复下载依赖。训练命令如下，长命令的反引号是 PowerShell 续行符；反引号后不要再留空格。

```powershell
py software/role_a/train.py train `
  --image-dir data/raw/isic2016/train/images `
  --mask-dir data/raw/isic2016/train/masks `
  --split runs/role_a_split/split.json `
  --output-dir runs/role_a_tinyunet `
  --epochs 40 --batch-size 4 --lr 0.001 --size 256 `
  --base-channels 8 --num-pools 3 --seed 42 `
  --device auto --num-workers 2 --augment
```

如果 py 调到另一套 Python，直接把每条命令的 py 替换为 .\.venv\Scripts\python.exe。Windows 的 DataLoader 多进程使用 spawn，worker 初始化函数要放在模块顶层，入口要由 if __name__ == '__main__' 保护；否则会出现无法序列化或递归启动。初学可先 --num-workers 0，正确后再加速。现有 train.py 已处理这个问题。

> **过关标准** 产生 best.pt、metrics.csv、metrics.json、config.json 和 split_used.json；metrics.csv 有 40 轮记录；最佳权重由验证 Dice 选择。绝不能根据测试结果回头挑 epoch。

第 7 章 评估：0.87 到底代表什么

7.1 Dice 与 IoU 从集合算起

把所有有效像素中预测为病灶的集合叫 P，真值病灶集合叫 T。交集 $|P\cap T|$ 是预测对的病灶像素。$Dice=2|P\cap T|/(|P|+|T|)$；$IoU=|P\cap T|/|P\cup T|$。用 TP/FP/FN 写，$Dice=2TP/(2TP+FP+FN)$，$IoU=TP/(TP+FP+FN)$。两者越接近 1 越好。

例：真值病灶 100 像素，预测也 100，其中重合 80。$Dice=160/200=0.8$；并集 $100+100-80=120$，$IoU=80/120\approx0.6667$。这解释为什么 Dice 与 IoU 不能拿同一数值直接比较。

若预测和真值都没有病灶，本项目把 Dice/IoU 定为 1；只有一方为空时定为 0；valid 区域全空则报错。先对**每张图**计算 Dice/IoU，再对图像取平均，让一张大照片与一张小照片各占一票；这与把所有像素合在一起算全局 Dice 不是一回事。

7.2 本次实际成绩与严格边界

输入经过 256×256 letterbox，只在 valid 区域按固定阈值 0.5 评价；这里的分数不是原图分辨率上的官方挑战评价，也不能直接与别人的论文分数横比。训练 40 轮后，验证 180 张 FP32 Dice/IoU 约 **0.8660/0.7800**；官方测试 379 张 FP32 约 **0.8756/0.7985**。量化软件仿真后的数字放在第 10 章。

这些验证数字受到第 4 章所述面积划分偏差影响。官方测试 379 张仍是单独评估，但最佳权重由偏向小病灶的验证集选出，故其选择过程也受影响。以下测试命令使用**单独登记测试文件名**的划分，且只读取冻结的 checkpoint：

```powershell
py software/role_a/train.py evaluate-test `
  --image-dir data/raw/isic2016/test/images `
  --mask-dir data/raw/isic2016/test/masks `
  --split runs/role_a_split_with_test/split.json `
  --checkpoint runs/role_a_tinyunet/best.pt `
  --size 256 --device auto `
  --output-json runs/role_a_tinyunet/test_metrics.json `
  --per-image-csv runs/role_a_tinyunet/test_per_image.csv
```

最小 10% 病灶的平均 Dice 在本次测试记录中约 0.747，其他图约 0.890。这提示小病灶难，但**不能用测试集据此挑新阈值或重选模型**；若要改进，应回到训练/验证集设计下一轮实验，再保留真正独立的最终测试。最差图 ISIC_0011152.jpg 的病灶面积约占原图 0.27%，说明平均分会遮住困难样例。

练习 6

写一个只接收两张布尔数组和 valid 掩膜的 dice_iou 函数。测试：完全相同=1；无交集=0；双方全空=1；valid 全空应报错；第 7.1 节例子应得到 0.8 和约 0.6667。不要把补边像素计入集合。

第 8 章 为什么要把浮点模型变成 INT8

8.1 从真实数到整数格子

训练时权重和特征通常是 FP32。一个 FP32 值占 4 字节；INT8 值占 1 字节，表达范围有限。**对称量化**约定 真实值≈整数 q×尺度 s、零点 z=0，本项目把整数夹在 [-127,127]。量化函数是 q=clip(round(x/s),-127,127)。反量化函数是 x□=q×s。尺度越小，单格越细，但能表示的最大真实值也越小。若需要覆盖绝对最大值 a，可取 s=max(a/127,1e-8)。

例：某张量最大绝对值 2.54，尺度 s=2.54/127=0.02。真实值 0.18 映射为 q=9，反量化又回到 0.18；真实值 0.19 则恰好是 9.5 格，最近偶数舍入给 10。很多不同浮点数会落到同一格，形成**量化误差**。如果真实值超出覆盖范围，夹到端点叫**饱和**。没有一个尺度能完全避免这两种误差。

本项目采用**激活每张量一个尺度**、**权重每输出通道一个尺度**：不同卷积输出通道的权重幅值可能很不同，给每通道独立尺度可减少小通道被大通道拖累。偏置和乘加用 INT32，防止大量 INT8 乘积相加时很快溢出。这个设计属于整数推理常见思路；具体边界、乘数精度和舍入是本项目自定接口，必须按本书的规则实现，不能想当然套用库默认量化。[来源 5]

8.2 四步推导整数卷积

普通卷积一个输出位置是 y=Σ(x_i*w_i)+b。若 x_i≈s_x*q_xi，某输出通道的权重 w_i≈s_w*q_wi，偏置取整数 q_b=round(b/(s_x*s_w))，那么整数累计 acc=Σ(q_xi*q_wi)+q_b，它代表的真实值约为 acc*s_x*s_w。如果下一层希望用尺度 s_y 保存输出，便要计算 q_y=round(acc*s_x*s_w/s_y)，最后夹到 [-127,127]。

软件浮点直接乘 s_x*s_w/s_y 可以，但硬件要更明确的整数规则。本项目定义 M=round((s_x*s_w/s_y)*2^24)，然后 q_y=clip(round_even(acc*M/2^24),-127,127)。这里 2^24=16777216，所以称 **Q24 固定小数点乘数**。乘法中间值用 INT64，除法用最近偶数舍入，再饱和到 INT8。若硬件用“普通右移截断”或“总向零舍入”，边界位置会和参考答案不同。

逐项手算例子

取 s_x=0.02，s_w=0.01，s_y=0.001。一项输入整数 q_x=9、权重整数 q_w=-3。真实偏置 b=0.008，整数偏置 q_b=round(0.008/0.0002)=40。累计 acc=9×(-3)+40=13，对应真实输出 13×0.0002=0.0026。乘数比为 0.0002/0.001=0.2，M=round(0.2×16777216)=3355443。于是 round_even(13×3355443/16777216)=3，输出 INT8 为 3，按输出尺度反量化是 0.003。误差 0.0004 是格子近似造成的。

另一边界：如果除法结果恰好 2.5，最近偶数得 2；3.5 得 4；-2.5 得 -2；-3.5 得 -4。**负数也要对称处理**，不能直接用语言的整数除法默认规则。项目的 software/role_a/quant_math.py 给出可独立阅读的参考算法。

```python
def round_divide_even(n, shift=24):
    sign = -1 if n < 0 else 1
    a = abs(n)
    q, rem = divmod(a, 1 << shift)
    half = 1 << (shift - 1)
    if rem > half or (rem == half and q % 2 == 1):
        q += 1
    return sign * q

def requantize(acc, multiplier_q24):
    return max(-127, min(127, round_divide_even(acc * multiplier_q24)))
```

正式实现还须检查 INT32 累计是否溢出、INT64 乘积是否溢出、尺度是否正且有限；不能让 NumPy 的整数溢出悄悄绕回去。权重张量按 OIHW 存放，偏置和累计按小端 INT32 存放。计算时每个输出通道用自己的 weight_scale 和 multiplier_q24。

练习 7

请不看代码，自己写一个单通道 3×3 INT8 卷积核：逐位置补零，按 Σ(q_x*q_w)+q_b 得到 INT32，再用上面的 Q24 规则输出 INT8。用第 5 章全 1 的例子，令尺度比恰为 1、乘数 2^24，应恢复中心 9、角 4、边 6。再测恰好半格的正负四种情况。

第 9 章 校准、完整前向和两类黄金向量

9.1 校准为什么只看训练集

要选激活尺度，先要知道真实运行时每层输入/输出大概落在哪个范围。**校准**就是运行已训练的 FP32 模型，记录若干图像在每层的最大绝对值，再据此定尺度。本次从 720 张训练图中按固定间距取 **32 张**；名单写入 model_spec.json。验证和测试图完全不参与校准，避免从考卷学习量化参数。

若只校准第一层，后面卷积的输入尺度仍未知。现有工具给各层记录尺度，算子之间可回到浮点等价表示，再按下一卷积入口的 input_scale 重新量化。skip 与 Concat 之后也一样。因此软件仿真是为**数值接口**定义的逐层量化网络，不是说 B 组必须在板卡上真的转回浮点；B 组可以用整数尺度变换实现同一个结果，但必须逐层核对。

```text
FP32 best.pt -> 32           ->    input/output scale
   FP32    ->       weight_scale -> INT8 weight
FP32 bias -> INT32 bias -> INT32 MAC -> Q24 -> INT8 output
```

运行导出前，确认 best.pt 已经训练完成且 split.json 是只含 train/val 的版本。运行：

```powershell
py software/role_a/quant.py export `
  --checkpoint runs/role_a_tinyunet/best.pt `
  --split runs/role_a_split/split.json `
  --image-dir data/raw/isic2016/train/images `
  --mask-dir data/raw/isic2016/train/masks `
  --output-dir runs/role_a_int8 `
  --calibration-images 32 --tile-size 8
```

9.2 黄金向量到底验证什么

假设 B 组做一个卷积模块。你给它固定输入、固定权重、固定偏置和**软件预先算出的正确输出**。如果板上的输出逐字节相同，该测试就验证了这一个模块对这组输入的乘法、加法、padding、舍入、饱和和字节布局。这组文件就是黄金向量。它不能单独证明所有输入、所有层和整个系统都正确，所以本项目同时给出两类。

小块黄金向量：每层取左上角 8×8 输入块，**独立**在块边界补零，输出 input.bin、weight.bin、bias.bin、acc_golden.bin、output_golden.bin。它适合验证独立卷积核，尤其方便手工检查边界。注意：它不等于从完整图卷积结果里裁一块，因为小块的右边/下边被当成零，而完整图的邻居不是零。

完整前向黄金向量：用同一张训练图片跑完整量化网络，每层保存 input_full.bin 和 output_full_golden.bin。它适合逐层查集成错误：第一层一致、第二层不同，问题多半出在两层之间的尺度转换或非卷积算子。小块与全图的目的不同，不能混用。

文件或字段	你该怎样理解	独立核验
model_spec.json	层名、shape、尺度、乘数、偏移	每层字段完整，15 层顺序固定
weights_int8.bin	所有层 OIHW 权重顺序拼接	121760 字节；offset 与 SHA256
bias.bin	所有层小端 INT32 偏置拼接	1412 字节；offset 与 SHA256
acc_golden.bin	小块卷积的 INT32 累计	按数学公式重算逐元素相等
output_golden.bin	小块重新量化后的 INT8	Q24 后逐字节相等

文件或字段	你该怎样理解	独立核验
output_full_golden.bin	完整网络该层 INT8 输出	集成链路逐层比较

weights_int8.bin 的 121760 是 121760 个 1 字节权重。bias.bin 的 1412 字节对应 1412/4=353 个 INT32 偏置。总参数量 121760+353=122113，和第 5 章一致。这是一个**跨文件自洽检查**，做交付的人必须会算。

9.3 二进制布局怎样由人手工检查

二进制文件不是文本，不要用记事本看。先从 JSON 读 shape、dtype 和 offset：一个 [O,I,3,3] 的 INT8 权重应占 O×I×3×3 字节；一个 [O] 的 INT32 偏置应占 4O 字节。offset 是它在拼接文件里从第几个字节开始。下一层 offset 应等于上一层 offset 加上一层长度，最后一个 offset 加长度应等于整个文件长度。小端 INT32 的字节 01 00 00 00 代表 1；FF FF FF FF 代表 -1。

用 Python 标准库亲手查一层的原始字节：

```python
from pathlib import Path
import struct
data = Path('runs/role_a_int8/bias.bin').read_bytes()
first_bias = struct.unpack_from('<i', data, offset=0)[0]
print('first int32 bias =', first_bias)
print('total bias bytes =', len(data))
```

<i 中 < 是小端，i 是有符号 32 位整数。要检查权重，用 numpy.frombuffer(data, dtype='int8') 并按 OIHW reshape；如果用无符号 uint8 读取，负权重会显示成 128-255，数值完全错。正式核验脚本如下，它**独立**重算小块整数卷积并检查哈希、偏移和黄金输出：

```powershell
py software/role_a/verify_export.py `
  --spec runs/role_a_int8/model_spec.json `
  --output-json runs/role_a_int8/verification.json
```

过关标准 15 层全部通过；改坏一个 weight.bin 字节，哈希检查或数值比对应失败。只检查 SHA256 不能证明数学正确；只看输出相等也不能证明文件未被调包，两种检查都需要。

第 10 章 对比 FP32 与 INT8，读懂量化误差

10.1 要比较同一批图、同一个权重

量化前后比较必须基于**相同 checkpoint、相同图像、相同预处理、相同有效区域和相同阈值**；改变其中一项，差异便无法归因到量化。本次 INT8 是 CPU 上的软件仿真，不是板卡输出。先在 180 张验证图比较：

```powershell
py software/role_a/quant.py compare `
  --checkpoint runs/role_a_tinyunet/best.pt `
  --spec runs/role_a_int8/model_spec.json `
  --split runs/role_a_split/split.json --role val `
  --image-dir data/raw/isic2016/train/images `
  --mask-dir data/raw/isic2016/train/masks `
  --output-json runs/role_a_int8/val_compare.json
```

正式测试对比需把 --split 换成 runs/role_a_split_with_test/split.json，--role 改为 test，两个数据目录改成官方 test，输出改为 test_compare.json。**先完成并冻结量化协议，再运行这一条**；测试分数只用于报告。

数据	FP32 平均 Dice / IoU	INT8 软件平均 Dice / IoU
验证 180 张	0.8660 / 0.7800	0.8578 / 0.7683
官方测试 379 张	0.8756 / 0.7985	0.8724 / 0.7944

官方测试 Dice 约下降 0.8756-0.8724≈0.0032，验证 Dice 约下降 0.0083。这不是“误差只有 0.32% 像素”的意思；Dice 是集合重合指标，差值是**指标点数**。CPU 与 CUDA 的 FP32 基准可能有约 10^-6 的微小差异，所以报告用四位小数。量化模型表现略低符合有限整数精度会引入误差的直觉，但不能仅凭这两行数字认定 FPGA 可用。

10.2 发生精度下降时怎么找原因

先确认文件/预处理一致，再看每层小块 INT32 累计是否正确。如果累计错，优先排查 NCHW/OIHW 顺序、padding、bias 位置、溢出；如果累计对而 INT8 输出错，排查 multiplier_q24、负数最近偶数舍入与饱和；如果单层都对而全网错，排查 ReLU、MaxPool、Upsample、Concat、skip 的顺序和重新量化尺度。最后才考虑算法层面的重新训练或换模型。**定位顺序从可直接验证的整数算术开始**，免得用调参掩盖接口 bug。

练习 8

人为把某层权重顺序从 OIHW 读成 HWIO，你预测小块黄金向量的哪个检查先失败？再把舍入由 ties-to-even 改为“半格一律远离零”，哪类输入会先出现差 1 的问题？答案应分别指向 INT32 累计、恰好半格的 requant 输出。

第 11 章 测试、文件清单与工程交接

11.1 测试不等于再算一遍正常输入

最有价值的测试会让程序碰到容易犯错的边界：缺失或重复掩膜必须报错；letterbox 的掩膜与 valid 必须同坐标；同种子划分必须稳定；U-Net 形状要严格对齐；预测/真值为空的指标要按约定；量化要测负数半格、端点饱和、溢出、错误 shape；导出要测文件长度、哈希和偏移。本项目在合成小图上运行 **92 项测试全部通过。**它证明这些受测规则在本机代码中成立，不证明对所有真实病灶都有好分数，也不证明 RTL/板卡通过。

```powershell
py -m unittest discover -s software/role_a/tests -v
```

源文件和交付文件各有用途：software/role_a/ 是可重建工具；runs/role_a_tinyunet/ 是本次训练现场记录；runs/role_a_int8/ 是量化导出工作目录；deliverables/role_a/ 是整理给 B 组看的冻结交付；deliverables/role_a_handoff.zip 把权重、配置、尺度、向量和报告打包，但**不包含几乎 1.7 GB 的官方原图和掩膜。**项目根目录的 .venv 是本机环境，也不该打进交付包。另一台电脑需要官方数据和自己的 Python 环境。

A 组交给 B 组什么	B 组需要回答什么
模型各层算子/shape/布局	RTL 中算子顺序和形状是否一一对应
INT8 权重、INT32 偏置、尺度/Q24	每层整数规则、字节序和饱和是否相同
小块及全图黄金向量	哪一层第一次出现逐元素差异
FP32/INT8 软件指标	板上输出是否与软件对齐；时延是否达标

11.2 容量预算怎样算，为什么它不是板级结果

默认网络 121760 字节 INT8 权重、1412 字节 INT32 偏置，合约 123172 字节（不含索引、缓冲、其他中间特征）。单个最大的 INT8 特征张量是拼接后的 [1,24,256,256]，占 24×256×256=1,572,864 字节，约 1.50 MiB。PYNQ-Z2 厂商规格给出片上 BRAM 630 KB、外部 DDR3 512 MB。[来源 6] 因此不能把这个峰值张量完整常驻 BRAM，B 组要设计 DDR 搬运、分块或流式缓存。但要实现整个 U-Net，skip 特征还会影响总峰值内存；**单张量 1.50 MiB 不是全系统内存上界。**这些都是算术估计，没有做 FPGA 综合、时序分析或上板实测。

练习 9

请算：若把 [1,24,256,256] 存成 FP32，要多少字节？答案 1,572,864×4=6,291,456 字节，约 6 MiB。再回答：为什么“权重很小”仍不代表所有特征图都能装入 630 KB BRAM？

11.3 项目目录导览：每个文件在流水线的位置

相对项目根目录的路径	用途
software/role_a/data.py	文件配对、letterbox、划分、Dataset
software/role_a/model.py	15 层 Tiny U-Net 和图结构检查
software/role_a/metrics.py	valid 内损失、Dice、IoU
software/role_a/train.py	划分、训练、测试与模型描述命令
software/role_a/quant_math.py	独立 INT32 卷积与 Q24 数学
software/role_a/quant.py	校准、导出、FP32/INT8 对比
software/role_a/verify_export.py	哈希、偏移、数值独立复核
deliverables/role_a/README.md	冻结结果、边界与 B 组接口说明

对照阅读法 先用本书写出自己的函数签名和 2-3 个测试，再打开对应源文件。比较的不仅是代码行数，还要问“错误输入会怎样”“日志记录了什么”“结果能否从文件独立复算”。

第 12 章 从空目录开始的独立实作实验

这一章是闭卷项目说明：你可在 scratch_a/ 另建目录，从头写自己的程序，不调用 software/role_a 里的函数。**可以使用 PyTorch、NumPy 和 Pillow**，它们是通用基础库；“纯人工实现”指你掌握算法、自己写流程和接口，而不是徒手重写深度学习框架。每个关卡都能在少量合成图片上先验证，然后再跑官方数据。建议先完成关卡 0-6 再动用显卡长期训练。

关卡 0：列实验合同

建立 experiment_contract.json，手填数据源 URL、任务 2016 Task 1、图像 256×256、RGB、letterbox、图像双线性/掩膜最近邻、seed 42、划分 720/180/379、网络通道 8/16/32/64、训练 40 轮、batch 4、学习率 0.001、阈值 0.5、量化范围 [-127,127]、Q24 ties-to-even。写完后你应该能回答每个数字是哪里来的：有的是官方数据事实，有的是本项目设计选择，有的是本次实验配置。不要把设计选择误称为医学定律。

关卡 1：数据核验与配对

输入四个官方 ZIP，输出完整 SHA256、解包后 900/379 对和错误清单。先写合成测试：删掉一张掩膜、加入同 ID 的重复文件、额外放一个孤立掩膜，各应报可读错误。再检查图像与掩膜宽高一致。若能从原图找到二值病灶区域，且 900/379 计数正确，才能继续。

关卡 2：letterbox 与 Dataset

写 letterbox(image,mask,size) 返回三张对象：照片、二值掩膜、valid。写 1024×768 → 256×192 → 上下 32 的几何测试；再写一张纯黑原图，证明不能根据黑像素推 valid。Dataset 的 __getitem__ 返回 float32 [3,S,S] 图、float32 [1,S,S] 掩膜、同形状 valid、文件名；取值范围分别是 [0,1]、{0,1}、{0,1}。

关卡 3：固定划分

先按 stem 排序，再用原始掩膜面积比做五档分层，每档按同一固定算法给验证集约 20%。保存 JSON 和 CSV，写 seed、文件名、train/val/test 的角色。两次运行比较文件哈希，必须一致。额外写测试断言五档验证数量都在合理范围，不能出现 90,90,0,0,0。训练入口遇到 test 角色时直接退出。说明患者级限制。若你采用修正后的算法，**必须重训与重评**，不能声称得到旧权重和旧分数。

关卡 4：模型与算术参考

自己写一个双卷积块，再写完整 Tiny U-Net。每创建一级就在纸上列 NCHW；先用随机张量前向，检查 [2,3,256,256] -> [2,1,256,256]；再独立写函数用 $Cout\times(Cin\times K^2+1)$ 计算总参数，并得到 122113。再估 MAC 得 675282944。若两种方法给出的数字不同，先查 Concat 输入通道和 head 的 bias。

关卡 5：损失、指标与训练

用第 6、7 章的小例子先写 valid_bce、soft_dice_loss、dice_iou，测试补边不影响损失。训练循环必须明确 train()/eval()、zero_grad/backward/step、每轮验证和按验证 Dice 保存最佳模型。先用几张自造黑白图做 1 轮流程自检，再用真实 720/180 数据跑 40 轮。记录配置、种子、最佳 epoch 和逐图验证指标。重训曲线不要求逐位相同，但应合理下降，且测试集从未进入训练。

关卡 6：独立测试

训练完成后冻结 checkpoint。官方 379 张测试图只运行评估，不在测试命令里实例化优化器。算每图 Dice/IoU、均值、中位数和空真值计数，写 JSON/CSV；报告一定写明“letterbox 后有效区域”和“阈值 0.5”。与本次 FP32 约 0.8756/0.7985 对照时，若你重训得到不同数字，先检查划分文件、预处理、模型结构、训练轮数和设备，不要修改测试阈值去凑分。

关卡 7：整数数学

不调用框架的量化算子，自写 symmetric_scale、quantize_int8、quantize_weights、quantize_bias、conv2d_int32、round_divide_even、requantize_int8。先过第 8 章手算例，再用一层随机小张量把你的输出与独立实现核对。特别要测试负数半格、极端饱和和整数溢出。然后写 15 层量化导出；校准名单只能来自 train。

关卡 8：字节级交接

把每层 OIHW 权重写 INT8，把 bias/acc 写小端 INT32，把 layer shape、scale、Q24 multiplier、offset 和 SHA256 写入 JSON。导出独立 8×8 小块及完整前向逐层向量。写另一个**不调用导出函数**的校验脚本，重新读取二进制，核查文件长度、offset、SHA256，并逐元素重算小块输出。故意改坏一个字节，验证脚本必须报错。只有这样才叫“可交接”。

关卡 9：比较并写诚实的结果

在同一权重、同一图像和同一协议上比较 FP32 与 INT8。报告验证集、官方测试集各自的 Dice/IoU，写差值和逐图困难样例。区分“训练真实跑过”“CPU 仿真通过”“FPGA 尚未实测”。打包应包含权重、JSON、二进制向量和报告，不包含原始官方数据与本机虚拟环境。此时你已经人工重做本次 A 组完整主线。

最终自测清单

- 我能不用看源码解释 NCHW、OIHW、logit、valid、Dice 和 Q24。
- 我能手算第一层 224 个参数、整个网络 122113 个参数。
- 我能解释为什么测试集不能用于选权重或校准尺度。
- 我能对一个 3×3 INT8 小卷积手算 INT32 累加与 Q24 输出。
- 我能定位“哈希错、累计错、requant 错、全网错”各自先看哪里。
- 我能说清楚本次 0.8756/0.8724 的评价空间和软件仿真边界。
- 我能在不引用 A 组现成函数的情况下生成并复核 B 组交接文件。

第 13 章 常见卡点与排错树

命令行和环境

提示找不到 torch：先运行 py -c "import sys; print(sys.executable)" 看你调用的解释器，再检查 .venv/Scripts/python.exe 是否有依赖。不要在多个 Python 中轮流安装包。**CUDA 显存不足**：先用 CPU 小图自检，正式训练可减少 batch，但这会改变训练配置和可能的结果，必须记录。**Windows DataLoader 报序列化错误**：先设 --num-workers 0，确认 worker 函数在模块顶层且入口有主模块保护。

数据和形状

找不到配对掩膜：核对 _Segmentation 后缀、扩展名大小写和目录层级。**图像与标签边缘错位**：检查 EXIF 处理、图像双线性/掩膜最近邻、两者使用同一缩放和补边坐标。**Concat 报尺寸不一致**：检查输入宽高是否可被 8 整除、MaxPool/上采样次数与 skip 层级是否对应。**出现非 0/1 标签**：别直接静默取整；记录文件名，再按约定阈值处理。

分数和量化

验证 Dice 高而测试低：先查划分与预处理是否一致，是否有数据泄漏或分布差异；不能靠测试集调参粉饰。**预测全背景**：检查掩膜方向、logit/阈值、损失对有效像素是否真的生效，以及训练是否更新参数。**INT8 某层完全不一致**：先用小块向量查布局与累计；累计对才查 Q24。**只差个别 1**：重点查负数和恰好半格的舍入，以及 ReLU 前后顺序。**哈希过、输出错**：文件传输完整，但算法或读取方式不一致。

自己解释一个“为什么”

你不应只背“先查数据”。请把问题链写出来：若掩膜配错，损失函数会告诉网络错误目标；网络会认真降低**错误目标的损失**；训练曲线也可能看起来正常。因此“损失下降”本身不是数据正确的证据。每层数据的正确性要靠独立检查，而不是靠最后一个平均分反推。

附录 A：纸笔答案与公式速查

卷积参数：Cout×(Cin×Kh×Kw+1)，末尾 1 是每输出通道一个 bias。第一层 8×(3×3×3+1)=224；第二层 8×(8×3×3+1)=584。**卷积 MAC**：Hout×Wout×Cout×Cin×Kh×Kw。**letterbox**：r=S/max(W,H)，新尺寸约为 (round(rW),round(rH))，居中补边。**Dice**：2TP/(2TP+FP+FN)。**IoU**：TP/(TP+FP+FN)。**sigmoid**：1/(1+exp(-z))。

量化：s=max(max_abs/127,1e-8)；q=clip(round_even(x/s),-127,127)；q_b=round_even(b/(s_x*s_w))；acc=Σ(q_x*q_w)+q_b；M=round_even((s_x*s_w/s_y)×2^24)；q_y=clip(round_even(acc*M/2^24),-127,127)。round_even(2.5)=2、round_even(3.5)=4、负数对称。

本次固定数字：官方数据 900 对训练、379 对测试；内部 train/val 720/180；输入 256×256 RGB；种子 42；网络 15 层卷积、122113 参数、675282944 MAC/图；训练 40 epoch、batch 4；校准 32 张训练图；INT8 权重 121760 字节、INT32 bias 1412 字节；单个最大 INT8 特征 1572864 字节。这些数字是本次配置和本机实物的验收点，不是任何 U-Net 必然如此。

附录 B：复现旧结果的精确细节与修正方案

B.1 历史 split.json 如何产生

旧版 data.py 先按 (病灶占比,stem) 排序 900 张训练掩膜；病灶占比由原始灰度掩膜中 >=127 的像素比例得到。每档长度 900//5=180，依次切成五档。用同一个 random.Random(42) **按档顺序**调用 shuffle。随后每档枚举，下标为偶数的项目追加到同一个 val 列表，奇数追加到 train；因此五档各贡献 90 张验证候选，val 暂时有 450 张。目标验证数是 round(900×0.2)=180，旧版把 val[180:] 移回 train，所以只留下前两档各 90 张。最后两个列表各自按 stem 排序写入 JSON。这一段是**旧结果可重复的准确说明，也是一个需要修正的工程缺陷。**

```python
ordered = sorted(rows, key=lambda x: (x.area_ratio, x.stem))
buckets = [ordered[i:i+180] for i in range(0, 900, 180)]
rng = Random(42)
train, val = [], []
for bucket in buckets:
    rng.shuffle(bucket)
    for i, row in enumerate(bucket):
        (val if i % 2 == 0 else train).append(row.stem)
for stem in val[180:]:
    val.remove(stem)
    train.append(stem)
train.sort(); val.sort()
```

这个偏差不是“数据泄漏”本身：测试图仍没有进入训练。但它使验证集只代表面积较小的病灶；用它选最佳 checkpoint 可能偏向这一部分，验证 Dice 的解释范围必须收窄。**不要在同一份旧报告里偷偷替换划分**：新划分会改变权重选择、量化校准名单及后续数字。正确做法是保留历史证据，在新实验编号下修正分配并重跑全流程。

B.2 下一轮怎样做真正均衡的五档划分

对本例每档 180 张，档内只抽 36 张验证、144 张训练即可，总计 5×36=180 验证、5×144=720 训练。对不能整除的其他样本数，先用每档 floor(档大小×0.2)，再按各档小数余数由大到小分配剩余名额；同余数时用固定档序决定，最后断言总数、无重复、各档占比接近。此法可重复，也比旧版更符合“分层”的本意；仍然不是患者级划分。下一轮还应在测试前核查每档 train/val 计数并写入报告。

B.3 校准名单与软件仿真的精确含义

旧版训练集按 split.json 的 stem 排序后共有 720 张。量化导出用 np.linspace(0,719,num=32,dtype=int) 选择 32 个**确定性下标**，并把其 stem 写入 model_spec.json。每个卷积的前后 hook 在这 32 张图上记录输入/输出最大绝对值，激活尺度按 max_abs/127 计算；权重尺度按每输出通道的最大绝对值计算。黄金向量来源是校准名单的**第一张训练图**。若你按随机抽样或只取前 32 张，尺度和二进制结果会不同。

软件量化网络在每层卷积入口把浮点等价输入按本层尺度变成 INT8。为方便在 CPU 上运行，当前 quant.py 用 FP64 的卷积运算执行**整数值**乘加，并在数值量级小于 2^53 的前提下得到精确整数和；之后转整数，做 Q24 舍入与饱和。独立的 quant_math.py 用 INT64 数组算小块并检查可装入 INT32，verify_export.py 再复核。因而“CPU 量化仿真输出符合整数协议”与“FPGA 上已经用了纯整数电路”仍是两件不同的事；后者需要 B 组 RTL 与板级测试。

> **学到的工程原则** **可重复**、**统计上合理**、**硬件可执行**是三个不同的检查目标。固定种子只解决第一项；五档计数审计解决第二项的一部分；逐层整数向量和上板实测才逐步检验第三项。

附录 C：出处、许可证与进一步阅读

本书的算法解释为教学性转述，数学例子为本书自编。实际命令与数值来自本项目 2026-09-23 的代码、runs/ 实验记录和 deliverables/role_a/ 冻结交付。网络结构采用公开的 U-Net 家族思路，代码独立实现。官方网址可能更新，重现实验请同时保存访问时间、ZIP 哈希和软件版本。

[来源 1] ISIC Challenge Datasets，2016 Task 1：训练 900 张 JPEG/900 张 PNG 掩膜；测试 379 张；页面标 CC0。https://challenge.isic-archive.com/data/。数据原始研究引用：Gutman 等，Skin Lesion Analysis toward Melanoma Detection，arXiv:1605.01397。

[来源 2] Ronneberger, Fischer, Brox，U-Net: Convolutional Networks for Biomedical Image Segmentation，2015：编码器/解码器和精确定位的原始论文。https://arxiv.org/abs/1505.04597。

[来源 3] PyTorch 官方 BCEWithLogitsLoss 文档：说明将 sigmoid 与 BCE 合并以提高数值稳定性。https://docs.pytorch.org/docs/stable/generated/torch.nn.BCEWithLogitsLoss.html。

[来源 4] PyTorch 官方 Reproducibility 文档：说明固定种子与跨设备/版本复现限制。https://docs.pytorch.org/docs/stable/notes/randomness.html。

[来源 5] Jacob 等，Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference，2018：整数推理量化的研究背景。本项目的 Q24 与黄金向量文件格式是项目自定规范。https://arxiv.org/abs/1712.05877。

[来源 6] TUL PYNQ-Z2 产品规格：片上 BRAM 与外部 DDR3 容量。https://www.tul.com.tw/images/PYNQ-Z2_PA_v2_pp_20201209_STD.pdf。

[来源 7] DeepSeek 官方 2026-09-10 更新日志：API 名称 deepseek-flash 对应 V4.1 Flash。https://api-docs.deepseek.com/updates/。

[来源 8] PyTorch 官方本地安装说明：不同系统与 CUDA 配置应按当前官方页面选择。https://pytorch.org/get-started/locally/。

项目内证据：data/raw/isic2016/provenance.json（四个 ZIP URL/SHA256）；software/role_a/README.md（运行命令）；runs/role_a_tinyunet/（训练与测试）；runs/role_a_int8/（量化与复核）；deliverables/role_a/README.md（冻结结果）；docs/09_A组开源资源核查与实施路线.md（现成资源筛选）；docs/prompts/A01_deepseek_基础实现.md（基础代码提示词）。

最后请记住：好的复现不是让某个分数“看起来对”，而是让**数据来源、每一步数学、每个文件、每个失败条件和最终边界**都能由另一个人独立检查。